Actividad de investigación Machine Learning

Introducción: el pipeline como una sola cadena

Antes de entrar a cada etapa, vale establecer la idea que las conecta a todas: un modelo de Machine Learning nunca es mejor que los datos y las decisiones que lo alimentan. Cada etapa de este documento existe *para* la siguiente — preparar mal los datos garantiza que ningún modelo, por sofisticado que sea, produzca resultados confiables. El orden importa tanto como el contenido de cada paso.

```
Datos crudos → Limpieza → Transformación → Selección de variables
  → Entrenamiento del modelo → Evaluación → Decisión de negocio
```

PARTE 1 — PREPARACIÓN DE LOS DATOS

1.1 Limpieza de datos

La limpieza de datos es el proceso de identificar y corregir —o decidir explícitamente cómo manejar— errores, inconsistencias y huecos en un dataset antes de usarlo para entrenar un modelo. No es una operación única: es una secuencia de decisiones donde el orden importa tanto como cada paso individual.

La regla más importante de toda la etapa: cualquier estadística que se calcule para limpiar (una media para imputar, un rango para detectar atípicos) debe calcularse únicamente con los datos de entrenamiento, nunca con el dataset completo. Calcularla antes de separar entrenamiento y prueba es una forma de fuga de datos (data leakage): el modelo evaluado más adelante daría métricas mejores de lo que realmente logrará en producción, porque el conjunto de prueba ya "vio" información derivada de sí mismo.

La secuencia recomendada de limpieza:

1. Perfilar antes de limpiar — entender la distribución de cada columna, el porcentaje de nulos, los tipos de dato, antes de tocar nada.
2. Resolver valores faltantes y duplicados (sección 1.2).
3. Unificar consistencia (formatos de texto, tipos de dato, categorías escritas de forma distinta pero equivalentes — "México", "MX", "Mexico" deben normalizarse antes de cualquier agregación).
4. Tratar valores atípicos (sección 1.3).

1.2 Valores faltantes y duplicados

Valores faltantes son celdas sin dato registrado. No todos los faltantes son iguales — clasificarlos correctamente determina si es seguro imputarlos o si hacerlo miente activamente:

Tipo	Significado	Qué hacer
MCAR (Missing Completely At Random)	Falta sin relación con ninguna otra variable	Seguro imputar con media/mediana, o eliminar la fila si son pocas
MAR (Missing At Random)	Falta relacionado con *otra* variable observada	Imputar usando esa variable (imputación por grupo)
MNAR (Missing Not At Random)	Falta relacionado con el propio valor que falta	Peligroso imputar — mejor crear un flag explícito de "faltante"

El caso MNAR es el más caro de ignorar: si los clientes con ingresos muy altos o muy bajos tienden a no reportar su ingreso, imputar con la media borra exactamente el patrón que hacía a ese dato informativo.

Duplicados se dividen en dos tipos con manejo distinto:

- Duplicado exacto: todas las columnas idénticas — casi siempre seguro eliminar, quedándose con una copia.
- Duplicado parcial: la misma entidad real con pequeñas diferencias (typos, formato distinto). Se detecta con *fuzzy matching* (comparar similitud de texto con un umbral de tolerancia) o definiendo claves compuestas de negocio (ej. email + fecha de nacimiento debería ser único). Eliminar de más borra información real; no eliminar infla artificialmente esa entidad en cualquier agregación o entrenamiento.

1.3 Valores atípicos (outliers)

Un valor extremo puede ser dos cosas muy distintas, y la decisión de qué hacer depende de cuál es:

- Un error de captura (una edad de 250 años) → corregir o eliminar.
- Un evento real pero raro (una transacción legítima diez veces el promedio) → no eliminar solo por ser extremo. En problemas de clases desbalanceadas (fraude, detección de enfermedades), los outliers suelen ser exactamente la clase minoritaria que el modelo necesita aprender a reconocer — eliminarlos automáticamente borra el objetivo mismo del proyecto.

La estadística (rango intercuartílico, desviaciones estándar de la media) puede *señalar candidatos* a outlier, pero decidir qué hacer con cada uno requiere criterio de dominio, no una regla automática de "todo lo raro se descarta".

1.4 Transformación y normalización

Una vez limpios, los datos numéricos casi siempre necesitan cambiar de escala antes de entrenar ciertos modelos (KNN, K-Means, regresiones, redes neuronales — no así los árboles de decisión y sus derivados, inmunes a la escala).

Técnica	Qué asume	Cuándo falla
Min-Max (rango 0-1)	No hay outliers extremos	Un solo valor extremo comprime todo lo demás en una franja diminuta
Estandarización (Z-score)	Distribución parecida a una normal	Distribuciones muy sesgadas rompen el supuesto
Robust Scaling (mediana + rango intercuartílico)	Hay outliers presentes	Opción más segura por default cuando no se ha verificado la distribución
Transformación logarítmica	Variable con cola larga (ingresos, precios)	Se aplica *antes* de escalar, no en vez de escalar

Codificación de variables categóricas (desarrollado a fondo en 1.5) también es una transformación: convertir texto a números de forma que el modelo pueda usarlos.

La misma regla de fuga aplica aquí: los parámetros de escalado (el mínimo, la media) se calculan solo con entrenamiento, y se aplican después al conjunto de prueba con esos mismos parámetros — nunca recalculados.

1.5 Variables categóricas

Las variables categóricas (país, tipo de producto) necesitan convertirse a números, y cómo se convierten importa:

- One-Hot Encoding: una columna binaria por categoría — correcto cuando las categorías no tienen orden real ("rojo", "azul", "verde").
- Ordinal Encoding: un número que respeta un orden real ("bajo"=1, "medio"=2, "alto"=3) — usarlo en categorías sin orden real es un error conceptual grave, porque le dice al modelo que existe una relación matemática que no existe en la realidad.

El problema de la cardinalidad alta: una variable con miles de categorías únicas (código postal) rompe One-Hot Encoding en la práctica (más columnas que filas). Alternativas: agrupar categorías poco frecuentes en "otros", o usar target encoding (reemplazar cada categoría por el promedio del target para esa categoría) — potente pero es una fuente casi garantizada de fuga de datos si se calcula sobre el dataset completo en vez de solo entrenamiento.

1.6 Selección de variables

Usar todas las columnas disponibles no es gratis: cada feature adicional aumenta el riesgo de **overfitting** (el modelo memoriza ruido en vez de aprender el patrón real) — el problema conocido como *maldición de la dimensionalidad*: con muchas features y pocos datos, el modelo tiene más formas de ajustarse a particularidades del entrenamiento que no se repiten en datos nuevos.

Tres familias de métodos para decidir qué variables conservar:

- **Filtro:** evalúan cada feature por su relación estadística con el target, de forma independiente — rápidos, ignoran interacciones.
- **Envoltura:** prueban distintos subconjuntos entrenando el modelo real con cada combinación — más precisos, más costosos.
- **Embebidos:** la selección ocurre dentro del propio entrenamiento (Lasso, importancia de variables de Random Forest) — el punto intermedio más usado en la práctica.

Multicolinealidad: cuando dos features están muy correlacionadas *entre sí* (no con el target), incluir ambas no agrega información nueva y puede volver inestable la interpretación de un modelo lineal.

1.7 Variables predictoras (features) y variable objetivo (target)

Las **features** (X) son las columnas que el modelo usa como entrada; el **target** (y) es lo que el modelo intenta predecir. Esta distinción **no es una propiedad fija de una columna** — depende de la pregunta que se hace y del **momento de corte**: una columna es feature válida solo si esa información existiría, en la realidad, *antes* del momento en que se necesita hacer la predicción.

Esta es exactamente la definición formal detrás de la fuga de datos: una columna que en realidad es consecuencia del resultado (ej. "número de llamadas de cobranza" para predecir impago — las llamadas ocurren *después* del atraso, no antes) tratada como feature es una fuga disfrazada de variable legítima.

PARTE 2 — MODELOS

2.1 ¿Qué es un modelo de Machine Learning?

Machine Learning es una rama de la Inteligencia Artificial donde un sistema aprende patrones a partir de datos de ejemplo, en vez de seguir reglas escritas explícitamente por un programador. En vez de decir "si A entonces B", se le dan muchos ejemplos y el sistema encuentra la relación entre entradas y resultado por sí mismo.

2.2 Aprendizaje supervisado y no supervisado

- Supervisado: los datos de entrenamiento ya traen la respuesta correcta (el target) — el modelo aprende a mapear entrada → salida conocida.
- No supervisado: no hay respuesta correcta — el modelo busca estructura oculta por su cuenta, sin que nadie le diga qué buscar.

El criterio real de cuál usar no es "cuál es mejor" — es qué información tienes disponible. Si ya sabes la respuesta histórica (¿este cliente canceló o no?), es supervisado. Si solo quieres descubrir patrones que no sabías buscar (¿qué grupos naturales de clientes existen?), es no supervisado.

2.3 Problemas de clasificación, regresión y clustering

Característica	Clasificación	Regresión	Clustering
Tipo de aprendizaje	Supervisado	Supervisado	No supervisado
¿Necesita target?	Sí	Sí	No
Qué predice	Una categoría	Un número continuo	Un grupo descubierto
Pregunta esencial	¿A qué clase pertenece?	¿Cuánto vale?	¿Qué grupos existen?
Ejemplo	¿Es spam o no?	¿Cuánto costará esta casa?	¿Qué segmentos de clientes hay?
Métricas	Accuracy, Precision, Recall, F1	MSE, RMSE, MAE, R^2	Silhouette score, inercia

2.4 Los seis modelos

Regresión Lineal

Predice un número continuo ajustando una línea recta (o hiperplano, con varias variables): $\hat{Y} = \beta_0 + \beta_1 X_1 + \ldots + \beta_n X_n$. Cada β representa cuánto cambia Y por cada unidad de esa X, manteniendo las demás constantes. Riesgo central: agregar variables siempre sube el R^2 aunque sean irrelevantes — usar R^2 ajustado para comparar modelos con distinto número de variables. Sensible a outliers y a extrapolar fuera del rango observado.

Regresión Logística

A pesar del nombre, es un modelo de clasificación: predice la probabilidad de pertenecer a una categoría, pasando la combinación lineal de las X por una función sigmoide que comprime cualquier resultado a un rango entre 0 y 1. El umbral de decisión (típicamente 0.5) no es fijo — moverlo cambia el balance entre Precision y Recall según qué tipo de error cueste más en el problema de negocio real.

Árboles de Decisión

Predicen haciendo una secuencia de preguntas sí/no sobre las variables, formando una estructura de árbol donde cada nodo elige la pregunta que produce la separación más pura posible entre clases (medida con el índice de Gini o entropía). Riesgo central: sin límite de profundidad, un árbol puede crecer hasta memorizar cada observación individual — el ejemplo más visual de overfitting. Se controla con pre-poda (`max_depth`, `min_samples_split`) o post-poda.

KNN (K-Nearest Neighbors)

Predice basándose en las K observaciones más parecidas ya conocidas — para clasificación, la clase más común entre esos vecinos; para regresión, su promedio. Es un "lazy learner": no construye ningún modelo durante el entrenamiento, todo el trabajo ocurre al predecir, calculando distancia contra cada punto guardado. Depende críticamente de que las variables estén normalizadas — sin eso, la variable de mayor magnitud numérica domina el cálculo de distancia sin importar su relevancia real.

Random Forest

Entrena muchos Árboles de Decisión distintos entre sí a propósito, y combina sus predicciones (voto o promedio). La diversidad entre árboles viene de dos mecanismos combinados: bagging (cada árbol ve una muestra aleatoria con reemplazo de los datos) y selección aleatoria de features (cada división considera solo un subconjunto aleatorio de variables). Ambos mecanismos son necesarios — sin ellos, el "bosque" colapsa en básicamente el mismo árbol repetido. Reduce drásticamente la varianza (inestabilidad) de un árbol individual sin aumentar mucho el sesgo.

K-Means

El primer modelo no supervisado de la lista: agrupa observaciones en K grupos (definidos por el usuario) según qué tan parecidas son entre sí, sin ningún target conocido. Proceso iterativo: asigna cada punto al centroide más cercano, recalcula los centroides como el promedio de sus puntos asignados, repite hasta converger. No existe un "accuracy" para evaluarlo — no hay respuesta correcta contra la cual medir, solo qué tan útiles resultan los grupos. Elegir K se apoya en el método del codo o el silhouette score, combinados con criterio de negocio.

Comparte la letra "K" con KNN sin ser el mismo tipo de problema: KNN es supervisado (necesita target); K-Means es no supervisado (no lo necesita).

2.5 Datos de entrenamiento, validación y prueba

Tres conjuntos, no dos:

- Entrenamiento: con los que el modelo aprende sus parámetros.

- Validación: usado para ajustar decisiones sobre el modelo (qué algoritmo, qué hiperparámetros) sin tocar el conjunto de prueba.
- Prueba: se toca una sola vez, al final, para medir la generalización real.

Por qué no basta con dos: si se usa el conjunto de prueba repetidamente para decidir qué modelo o hiperparámetro es mejor, el test deja de ser una medición honesta — se contamina indirectamente por el propio proceso de selección de modelo.

Validación Cruzada (Cross-Validation) resuelve una limitación de tener una sola partición fija: divide los datos en K partes ("folds"), entrena K veces (cada vez con una parte distinta como validación), y promedia el resultado — dando una estimación mucho más estable que depender de una sola división que pudo salir, por azar, poco representativa. Stratified K-Fold es la variante necesaria cuando las clases están desbalanceadas, asegurando que cada fold mantenga la proporción real de clases del dataset completo.

2.6 Overfitting y underfitting

- Overfitting (sobreajuste): el modelo memoriza el ruido específico de entrenamiento — funciona muy bien ahí y notablemente peor en datos nuevos.
- Underfitting (subajuste): el modelo es demasiado simple para capturar ni siquiera el patrón real — funciona mal en ambos conjuntos por igual.

Cómo se detecta: comparando el error de entrenamiento contra el de validación. Error de entrenamiento bajo + error de validación alto → overfitting. Ambos altos y parecidos → underfitting. Ambos bajos y parecidos → el punto que se busca.

PARTE 3 — IMPLEMENTACIÓN Y EVALUACIÓN

3.1 ¿Qué significa evaluar un modelo?

Medir qué tan bien las predicciones se parecen a la realidad — pero "qué tan bien" no tiene una sola respuesta: depende de si el problema es de clasificación o regresión, y de cuál tipo de error importa más para el negocio específico. No existe una métrica universal.

El riesgo central: Accuracy (% de aciertos) es la métrica más intuitiva y la que más engaña con clases desbalanceadas — un modelo que predice siempre "no" en un problema donde solo el 5% de los casos son "sí" obtiene 95% de accuracy sin haber aprendido nada útil.

3.2 Matriz de confusión (la base de todo lo demás)

	Realidad: Positivo	Realidad: Negativo
Predicción: Positivo	Verdadero Positivo (TP)	Falso Positivo (FP)

	Realidad: Positivo	Realidad: Negativo
Predicción: Negativo	Falso Negativo (FN)	Verdadero Negativo (TN)

Cada métrica de clasificación es una forma distinta de combinar estas cuatro cantidades — no fórmulas sueltas para memorizar, sino distintas preguntas sobre la misma tabla.

3.3 Métricas de clasificación

- Accuracy `= (TP+TN) / Total` — % de aciertos totales, ciego al tipo de error.
- Precision `= TP / (TP+FP)` — de lo que dije que era positivo, ¿cuánto lo era realmente? (mide falsas alarmas)
- Recall `= TP / (TP+FN)` — de lo que realmente era positivo, ¿cuánto detecté? (mide casos que se escaparon)
- F1-Score `= 2×(Precision×Recall)/(Precision+Recall)` — media armónica de ambas, un solo número que resume el balance.

El trade-off central: subir Recall (bajando el umbral de decisión) casi siempre baja Precision, y viceversa — ambas dependen del mismo umbral. La decisión de hacia qué lado inclinarse depende de qué error cuesta más en el negocio real: un falso negativo en detección de fraude (dejar pasar un fraude real) suele costar más que un falso positivo (revisar una transacción legítima de más).

Extensión natural — Curva ROC y AUC: en vez de fijar un umbral, la curva ROC muestra el comportamiento (Tasa de Verdaderos Positivos vs. Tasa de Falsos Positivos) a través de todos los umbrales posibles. AUC resume esa curva en un número (0.5 = azar, 1.0 = perfecto). Riesgo importante: AUC es ciego al desbalance de clases igual que Accuracy — con eventos muy raros, conviene la curva Precision-Recall en su lugar.

3.4 Métricas de regresión

- MAE (Mean Absolute Error) — promedio del error absoluto, trata todos los errores por igual.
- MSE (Mean Squared Error) — promedio del error al cuadrado, penaliza desproporcionadamente los errores grandes.
- RMSE (Root Mean Squared Error) — raíz de MSE, devuelve el error a las unidades originales (más fácil de comunicar).
- R^2 — proporción de la variación de Y que el modelo explica, comparado contra simplemente predecir el promedio siempre. Riesgo ya visto en 2.4: sube con cada variable agregada, use R^2 ajustado para comparaciones justas.

Criterio de elección: MAE cuando se quiere una medida robusta ante errores ocasionales grandes; MSE/RMSE cuando esos errores grandes son especialmente indeseables y se quiere que el modelo los penalice más durante el entrenamiento.

Cierre: el pipeline completo, de principio a fin

```
1. Dataset crudo
2. Limpieza (faltantes, duplicados, outliers, consistencia)
3. Transformación y normalización + codificación de categóricas
4. Selección de variables (features vs. target ya definidos)
   ↓
5. Split: entrenamiento / validación / prueba
6. Elegir tipo de problema: clasificación, regresión o clustering
7. Entrenar el modelo (ajustando hiperparámetros con validación
   o Cross-Validation, nunca con el conjunto de prueba)
8. Evaluar con las métricas correctas para el tipo de problema
9. Verificar overfitting/underfitting comparando train vs. validación
10. Medición final honesta con el conjunto de prueba, una sola vez
```

Cada fallo típico de un proyecto de Machine Learning se explica por saltarse o hacer mal alguno de estos diez pasos — no por elegir el algoritmo equivocado. La mayoría de los problemas reales de un modelo que "funciona en pruebas y falla en producción" viven en los pasos 1 a 5, no en el 6 o 7.

Notas de profundización (para quien quiera el mecanismo completo de cada tema)

Cada sección de este documento es una síntesis. El desarrollo completo, con ejemplos, riesgos y criterio detallado, vive en:

Preparación de datos: Data Set · Data Cleaning · Valores Faltantes y Duplicados · Transformación y Normalización · Features, Target y Selección de Variables · Data Leakage

Modelos: Machine Learning · Regresión Lineal · Regresión Logística · Árboles de Decisión · KNN (K-Nearest Neighbors) · Random Forest · K-Means

Implementación y evaluación: Implementación y Evaluación · Curva ROC y AUC · Validación Cruzada (Cross-Validation)

Contexto adicional relacionado: Correlación no es causalidad · Predicción de Churn (caso de aplicación de negocio completo)